Lecture 1: What Is an Interactive Theorem Prover?

Stefano M. Nicoletti

1 Lecture 1: What Is an Interactive Theorem Prover?

1.1 Aim of the Lecture

This lecture introduces the general idea of an interactive theorem prover and then presents Lean as one specific system of this kind. The aim is not yet to learn many tactics or to formalize complex examples. The aim is to understand what it means to build a proof interactively, what the user sees during that construction, and why a system such as Lean can be useful for analyzing arguments, programs, mathematical proofs, and scientific models.

The lecture has three parts:

- a theoretical part: what is a theorem prover?
- a classroom demonstration;
- a set of exercises.

1.2 What Is an Interactive Theorem Prover?

A theorem prover is a system that helps build and check formal proofs. Some theorem provers are mostly automatic: the user states a problem and the system tries to find a proof. An interactive theorem prover, by contrast, puts the interaction between user and system at the center (Source: Theorem Proving in Lean 4, Propositions and Proofs).

In an interactive system the user writes definitions, statements, and proof steps. The system responds by showing the current state of the proof, or *proof state*, and accepts only steps that respect the formal rules of the system. The complete proof is accepted as correct because it can be checked mechanically: it is a *machine-checkable proof*.

A proof assistant does not replace human judgment. It does not decide by itself what the right formalization of a problem is, nor does it establish whether the premises of an argument are empirically or historically well founded. Its contribution is circumscribed: once statements, definitions, and assumptions have been fixed, it checks whether a given formal construction really proves what it claims to prove.

1.3 Lean

Lean is a specific interactive theorem prover. It is also a programming language. This double nature is central: Lean lets us write computational objects, mathematical structures, logical statements, and proofs in one formal environment (Source: de Moura and Ullrich, Lean 4).

In Lean we can:

- define objects;
- formulate statements;
- build proofs;
- follow the development of a proof interactively;
- formally check the result.

For this first lecture, it is enough to keep one idea fixed: Lean does not check an informal text, but a formal object. If the proof is accepted, the system has checked that the object we constructed has the type required by the statement. But what does this mean?

1.4 Objects and Types

In Lean every object, or *term*, has a type. The type tells us what kind of object we are dealing with and guides what it makes sense to do with it (Source: Theorem Proving in Lean 4, Dependent Type Theory).

For example, objects that represent natural numbers, of type *Nat*, can be used in computational reasoning: we can add, multiply, define recursive functions, or prove arithmetic properties. Objects that represent propositions, of type *Prop*, instead enter reasoning about truth and falsity.

The type of an object therefore informs the way Lean can treat it. If we have a numeric object, Lean expects operations and properties appropriate for numbers. The systematic relation between types, terms, propositions, and proofs will be addressed in the next lectures.

1.5 Interactive Proofs

In an interactive proof we can observe the current state of the proof. This state is often called the *proof state*.

The proof state contains at least two fundamental pieces of information:

- the *goal*, namely what remains to be proved at that point;
- the local *hypotheses and assumptions* available at that point.

At the beginning, the goal coincides with the target of the statement. During the proof it can change: it can be transformed into a simpler goal, split into several goals, or closed when exactly the required object is supplied. A proof is complete when no open goals remain.

Hypotheses are the information we can use at a given moment in the proof. Some come directly from the statement. Others can be introduced during the proof. Some are local, for example inside one branch of a proof by cases.

1.6 Tactics

A *tactic* is a command that transforms the proof state. It is not the final proof in the most fundamental sense, but it is a practical way to build one interactively.

A tactic can, for example:

- introduce a new assumption;
- apply an available rule;
- split a problem into cases;
- produce an intermediate result;
- close a goal when an object of the required type is available.

The pedagogical perspective of the first lecture is this: an interactive proof is a controlled sequence of transformations of the proof state. The user proposes a step; Lean updates the goal and the context, or rejects the step if it is not formally valid.

1.7 Why Use a Theorem Prover?

1.7.1 Argumentation

A theorem prover forces us to distinguish premises, inferential steps, and conclusion. This is useful even when the main interest is not mathematical. In ordinary arguments many premises often remain implicit, or the transition from the premises to the conclusion is left to the competence of the reader.

A formalization makes the structure visible:

- which assumptions are being used;
- which conclusion is being targeted;
- which inferential step connects premises and conclusion.

Of course, this does not solve every problem in argumentation. Lean does not decide whether a premise is acceptable, whether a source is reliable, or whether the formalization really captures the informal argument. But it does check the formal part of the inferential steps.

1.7.2 Software Correctness

A program can compile, pass many tests, and still violate the intended specification. Tests show that some cases work. A formal proof can instead establish that a desired property holds for all possible executions of a given program.

Formal verification is relevant when we want stronger guarantees than tests alone can provide: compilers, protocols, critical systems, algorithms, and software infrastructures may require explicit and checkable properties (Source: VU Amsterdam, Logical Verification).

1.7.3 Mathematics

In mathematics, a formalized proof makes the relation between definitions, lemmas, and conclusion checkable. Every step must be justified relative to explicit rules. This does not remove the value of informal mathematical writing, but it adds a layer of mechanical control.

Lean is especially important because of its mathematical library, `mathlib`, built collaboratively. Once a result has been formalized, it can be reused by others as part of a library of verified mathematics (Source: The mathlib Community, The Lean Mathematical Library).

1.7.4 Physics

In physics, a theorem prover can help make the assumptions of a model explicit, check mathematical or logical steps, and clarify which assumptions belong to a derivation. It does not decide whether a physical model is empirically adequate. But it can help separate the empirical question from the formal one: given a certain model and certain assumptions, what follows?

Physlib is a community project for formalizing results in physics in Lean 4. It is a useful example of how formalization can become collaborative work outside pure mathematics as well (Source: Physlib).

1.8 What Does Lean Check?

Lean checks the correctness of a formal construction relative to a formalized statement. Lean does not decide, however:

- whether an empirical premise is true;
- whether a historical source is reliable;
- whether a physical model is adequate for a phenomenon;
- whether a formalization captures everything we wanted to say;
- whether a proof is pedagogically clear.

Lean checks that, given certain definitions and assumptions, the formal conclusion follows according to the rules of the system. Mechanical checking does not eliminate human judgment, but it moves many errors or oversights from informal reading to the explicit checking of a formal object.

1.9 How Can We Trust Lean?

Trust in Lean depends on how the system is built. The central idea is to reduce the part of the system we have to trust.

Lean can offer complex interfaces, sophisticated tactics, and huge libraries. However, the final result must be checked by a relatively small kernel. The *kernel* is the part of the system that checks whether the constructed proof is correct. For this reason it belongs to the *trusted computing base*: it is the part we have to trust for verification to have value (Source: de Moura and Ullrich, Lean 4).

The principle of the small kernel is pragmatic: if the system contains many complex tools, we want errors in those tools not to be enough, by themselves, to accept a wrong proof.

Trust in Lean is also supported by the open and community-based character of the system. Kernels, libraries, and tools can be publicly discussed, inspected, corrected, and compared with independent implementations or checkers. Projects such as *Lean4Lean* show a further

direction: using formal tools to verify parts of the metalanguage and the formal checker itself (Source: Carneiro, Lean4Lean).

1.10 Argument and Good Argument

We begin our journey into Lean by formalizing some arguments. But first: an argument connects premises and conclusion through an inference (Source: Stanford Encyclopedia of Philosophy, Argument and Argumentation).

- The premises are what we start from.
- The conclusion is what we want to support.
- The inference is the transition from premises to conclusion.

A good argument requires at least three dimensions.

Dimension	Question
Acceptability	Are the premises true, credible, or defensible?
Validity	Does the conclusion follow from the premises?
Soundness/correctness	Are the premises acceptable or true, and is the inference valid?

Validity concerns the form of the inferential transition. An argument can have plausible premises but an invalid inference. Soundness, or correctness, requires both: true premises and a valid inference.

Lean enters mainly at the formal dimension of validity. It can check that a conclusion follows validly from certain formalized premises. It cannot, by itself, establish that those premises are acceptable in the philosophical, historical, scientific, or empirical context in which we use them.

1.11 Ordinary Language and Logical Form

Before formalizing, we need to recognize some recurring forms in ordinary language. In this lecture we restrict ourselves to three fundamental connectives (Source: MIT OpenCourseWare, Logic I).

Ordinary language	Logical form
We have both pieces of information	AND, namely $\wedge$
We have at least one of two alternatives	OR, namely $\vee$
If a condition or hypothesis holds, then a consequence follows	IMPLIES, namely $\to$

1.12 Installing Lean

To work with Lean we will use VS Code with the Lean 4 extension. The standard recommended by the official installation page is (Source: Lean official install page):

- install VS Code;
- install the Lean 4 VS Code extension;
- follow the setup guide opened by the extension;
- create a new file with the `.lean` extension from the VS Code menu File > New File.

The official page remains the operational reference for the current setup.

1.13 Classroom Demonstration

The file used for live coding in the lecture is `Classroom.lean`, which you can find on the reference website. The purpose of the file is to see the construction of the proof while the proof state changes. The examples proceed gradually: first we check types and expressions, then we build elementary proofs, then we use ∧ and → in a first reusable schema.

At the beginning we see some orientation commands:

```
#check Prop
#check Nat
#check Float

#eval 3 + 1
#eval 3.5 + 2
```

`#check` asks Lean for the type of an expression. `#eval` instead asks Lean to evaluate a computational expression. At this point we are not yet proving theorems: we are observing that Lean always works with typed objects.

The first proof exercise is minimal:

```
-- Natural language: if we assume that it rains, then we can conclude that it
   rains.
example (Rains : Prop) (hRains : Rains) : Rains := by
  sorry
```

`sorry` is a placeholder: it tells Lean to temporarily accept a hole in the proof. The complete version is:

```
-- Natural language: if we assume that it rains, then we can conclude that it
   rains.
example (Rains : Prop) (hRains : Rains) : Rains := by
  exact hRains
```

The tactic `exact` closes the goal by providing exactly a term of the required type. The goal is `Rains`; in the context we have `hRains : Rains`; therefore `exact hRains` is enough.

The next step introduces an implication:

```
-- Natural language: if it rains, then it rains.
example (Rains : Prop) : Rains → Rains := by
```

```
  intro h -- intro informativeName, any name that helps us remember the
   hypothesis
  exact h
```

The tactic `intro` is used when the goal is an implication. To prove `Rains → Rains`, we temporarily assume `Rains` and call this hypothesis `h`. At that point the goal becomes `Rains`, which is already available as `h`. So we can close the proof with `exact`.

Now we move to a conjunction:

```
-- Natural language: if it rains then I take the umbrella, and it rains, then
-- this implies that I take the umbrella.
example (Rains TakeUmbrella : Prop) :
    ((Rains → TakeUmbrella) ∧ Rains) → TakeUmbrella := by
  intro hypotheses
  have hRainsUmbrella := hypotheses.left
  have hRains := hypotheses.right
  have hTakeUmbrella := hRainsUmbrella hRains
  exact hTakeUmbrella
```

Here `hypotheses` is a conjunction. The left part is the rule `Rains → TakeUmbrella`; the right part is the proof of `Rains`. The tactic `have` introduces an intermediate result into the context. First we separate the left part of the conjunction with `.left`, placing it in `hRainsUmbrella`; then we do the same with the right part and `hRains`. We then apply `hRainsUmbrella` to `hRains` and obtain `TakeUmbrella`.

The next step is to give a name to the reasoning schema so that it can be reused in other proofs:

```
-- We encode, verify, and use the valid reasoning schema. Braces mark implicit
-- arguments: Lean can often infer them from the type of the hypotheses.
-- Natural language: if P is true and Q follows from P, then Q is true.
theorem lecture01_modus_ponens
    {P Q : Prop}
    (hPQ : P → Q) (hP : P) :
    Q := by
  exact hPQ hP
```

This is modus ponens. The braces in `{P Q : Prop}` mark implicit arguments: Lean can often infer them from the type of the hypotheses. Once the theorem has been proved, we can use it by applying it to a concrete case:

```
example (Rains TakeUmbrella : Prop)
    (hRains : Rains)
    (hRainsUmbrella : Rains → TakeUmbrella) :
    TakeUmbrella := by
  exact lecture01_modus_ponens hRainsUmbrella hRains
```

Here `exact` closes the goal with the application of the general theorem to the concrete case. The important point is the order of the arguments. In `lecture01_modus_ponens`, `{P Q : Prop}` are implicit: Lean infers them from the types of the hypotheses. The

first explicit arguments are therefore, in order, `hPQ : P → Q` and `hP : P`. This is why we write `lecture01_modus_ponens hRainsUmbrella hRains`: first the rule `Rains → TakeUmbrella`, then the proof of `Rains`. If we reverse them, Lean receives a proof where it expects a function, so the application does not have the right type to close the goal.

1.14 Tactics, Commands, and Input Shortcuts

In this lecture we used these commands and tactics:

- `#check`: shows the type of an expression;
- `#eval`: evaluates a computational expression;
- `intro`: introduces an assumption when the goal is an implication;
- `exact`: closes the goal by providing a term of the required type;
- `have`: introduces an intermediate result with a name.

To type the logical symbols in VS Code with the Lean 4 extension, these shortcuts are useful:

Shortcut	Symbol	Use
`\r`	→	implication
`\and`	∧	conjunction
`\or`	∨	disjunction
`\iff`	↔	if and only if
`\l`	←	left arrow

1.15 Exercises and Solutions

The files with exercises and solutions (`Exercises.lean` and `Solutions.lean`) are available on the reference website.

1.16 Sources and Further Reading

- Lean official install page: https://lean-lang.org/install/.
- Jeremy Avigad, Leonardo de Moura, Soonho Kong, Sebastian Ullrich, *Theorem Proving in Lean 4*, chapter ”Propositions and Proofs”: https://docs.lean-lang.org/theorem_proving_in_lean4/Propositions-and-Proofs/.
- Jeremy Avigad, Leonardo de Moura, Soonho Kong, Sebastian Ullrich, *Theorem Proving in Lean 4*, chapter ”Dependent Type Theory”: https://lean-lang.org/theorem_proving_in_lean4/Dependent-Type-Theory/.
- Leonardo de Moura and Sebastian Ullrich, ”The Lean 4 Theorem Prover and Programming Language”: https://lean-lang.org/papers/lean4.pdf.
- Mario Carneiro, ”Lean4Lean: Towards a Verified Typechecker for Lean, in Lean”: https://arxiv.org/abs/2403.14064.
- The mathlib Community, ”The Lean Mathematical Library”: https://arxiv.org/abs/1910.09336.

- Physlib, physics in Lean 4: https://github.com/leanprover-community/physlib.
- Stanford Encyclopedia of Philosophy, "Argument and Argumentation": https://plato.stanford.edu/archives/sum2022/entries/argument/.
- MIT OpenCourseWare, Logic I: https://ocw.mit.edu/courses/24-241-logic-i-fall-2009/.
- VU Amsterdam, Logical Verification: https://github.com/vu-logical-verification/logical_verification_2025.